Problema de la Mochila (Primera visita)

Tenemos una mochila con una capacidad W (peso, volumen). Hay elementos a guardar. Cada elemento tiene un peso y un valor. Queremos maximizar el valor de lo que nos llevamos sin pasarnos de la capacidad.

Vamos a plantear 3 posibles algoritmos que se nos ocurren.

Alternativa 1: Ordenamos de mayor a menor valor

Recordamos, nuevamente, que ordenar en sí mismo no es algo greedy, sino tan solo una optimización. Nuestra regla greedy va a ser obtener el de mayor valor de los elementos que queden, y si entra en la mochila (considerando lo ya guardado), lo pondremos, y sino lo descartaremos. Ordenaremos para que obtener el de mayor valor sea hecho luego en $\mathcal{O}(1)$ en cada iteración.

¿Cuál es el óptimo local? Agarramos el elemento que más nos aumente el valor acumulado en esta iteración.

Implementación:

```python
# Tomemos por ejemplo a los elementos como tuplas (nombre, valor, peso)
def mochila(elementos, W):
    ordenados = sorted(elementos, key=lambda elem: elem[1], reverse=True)
    resultado = []
    peso_acumulado = 0
    for elem in ordenados:
        if peso_acumulado + elem[2] <= W:
            peso_acumulado += elem[2]
            resultado.append(elem)
    return resultado
```

¿Complejidad? Esperamos que sea evidente que es $\mathcal{O}(n \log n)$ por el ordenamiento (la iteración luego es simplemente lineal).

¿Es óptimo? No, y tenemos un contraejemplo sencillo:

```
W = 10
Elementos = [(10, 10), (9, 9), (8, 1)]
```

En este caso nuestro algoritmos nos dará como resultado solo el primer elemento, cuando los otros 2 suman más. Esto es porque en ningún momento el algoritmo presta atención a los pesos.

Alternativa 2: Ordenar de menor a mayor peso

En este caso nuestra regla greedy sería agarrar al elemento de menor peso, y si entra guardarlo (si no entra, podemos parar, nada siguiente podrá entrar de todas formas).

El problema aquí es el óptimo local... En sí, podríamos decir que este algoritmo no tiene nada que ver con lo que buscamos (optimizar el valor obtenido) porque ni siquiera mira el valor. Se podría argumentar "vemos de poner la mayor cantidad de elementos", pero no es lo buscado en absoluto, así que no hay relación en sí entre lo que hace este algoritmo y el problema, por lo que es cuanto menos discutible que sea realmente un algoritmo greedy para este problema.

Tal es así que encontrar un contraejemplo es extremadamente sencillo:

```
W = 10
Elementos = [(100000, 10), (9, 9), (8, 1)]
```

Al no ver el valor de ninguna forma, puede estar totalmente fuera de cualquier tipo de cercanía con el óptimo real.

Alternativa 3: Una proporción v/w

Ahora ordenaremos, de mayor a menor, por la relación entre el valor y el peso (valor/peso, valor por unidad de peso) de cada elemento. Digamos, el "mejor calidad/precio".

La regla greedy es clara, obtener el de mejor relación valor/precio, si entra lo guardamos, sino descartamos y seguimos adelante. El óptimo local es encontrar dicho elemento porque es el que nos asegura optimizar cuanto valor estamos dedicando a cada espacio que tenemos. Es importante ver que este algoritmo devuelve la solución óptima en los dos casos anteriores que se plantearon como contraejemplos.

```python
# Tomemos por ejemplo a los elementos como tuplas (nombre, valor, peso)
def mochila(elementos, W):
    ordenados = sorted(elementos, key=lambda elem: elem[1]/elem[2], reverse=True)
    resultado = []
```

```
    peso_acumulado = 0
    for elem in ordenados:
        if peso_acumulado + elem[2] <= W:
            peso_acumulado += elem[2]
            resultado.append(elem)
    return resultado
```

Notar que la solución es extremadamente similar a la alternativa 1, solo cambió el criterio de orden, la complejidad es la misma.

¿Es óptimo? No. Como siempre, no necesariamente algo bueno en relación calidad/precio es lo mejor, porque algo muy malo pero de muy bajo costo nos puede estar usando espacio precioso. En base a eso pensamos el contrajemeplo:

```
W = 10
Elementos = [(3, 1), (10, 10)]
```

La relación 3 a 1 del primero hace que solo nos quedemos con este.

Cuestiones importantes de las propuestas

Sacando la segunda propuesta que es exageradamente mala (podríamos decir que lo mala que es se basa en que justamente no es realmente greedy, no podemos enunciar un óptimo local relacionado al global que nos interesa), las otras 2 tienen algo interesante: cuando una se comporta mal, la otra se comporta bastante bien (notar contraejemplos de cada una, la contraria lo resuelve). Entonces una idea sería ejecutar ambos algoritmos, y quedarnos con la solución de más valor acumulado.

Esto no necesariamente es óptimo, como hemos podido ver empíricamente en clase con la demo (pueden ver el código en el repositorio de ejemplos en el directorio `mochila`).

El problema original que planteamos es conocido como "Problema de la mochila 0/1", es decir, de cada elemento ponemos un elemento o no lo ponemos. Hay otras versiones de este problema. Una de ellas es el de "Mochila Fraccionada". En este caso, se pueden poner proporciones de elementos (como máximo, el elemento entero). Es decir, podemos poner un 0.5 del primer elemento, 0.2 del segundo, todo el tercero, etc... En este caso, el algoritmo de calidad/precio es óptimo. Ordenamos de la misma forma, guardamos mientras entre, y cuando un elemento no entre, guardamos tanto como quede de espacio (es decir, solo uno irá realmente fraccionado, que es el último, el de menor relación valor/precio, y luego podemos parar porque la mochila quedó llena).

Pero entonces, ¿se puede resolver óptimo el problema 0/1?

Con un algoritmo greedy, lamentablemente no (si alguien encuentra la forma, nos avisan, prometemos un 10 en la materia en ese caso). Revisitaremos este problema múltiples veces para ver de resolverlo (de forma óptima) por otros medios y técnicas.